

PROJECT BRIEF

☐ Scenario – Seeking ways to attack the network

Simulate attacks on sensitive configuration

Triggering defense mechanisms to ensure network flow (like **NMAP** and **Certificate Inspection**)

☐ Disrupting Server Flow

DNS poisoning

NTP blocking

☐ Attacking Network Overflow

ARP poisoning

ICMP flood (endless ping flood)

☐ Wireless Access

Adding firewall instructions using **NETSH** to block/poison network protocols

ARP Poisoning

ISSUE DISCOVERED

Process

CMD: arp -a => Show MAC table (upper screenshot)

CMD: arp -s 192.168.1.1 aa-bb-cc-dd-ee-ff => Statically define the router's MAC address (the lower screenshot)

CMD: arp -d => Fixing the arp table

Result

The router 192.168.1.1 has its MAC address defined statically in the interface

192.168.56.1

Very interesting finding here — the poison only appeared on the **VirtualBox interface** (192.168.56.1), not the Wi-Fi interface. This is because:

arp -s without specifying an interface applies to whichever interface **Windows** chooses.

“Teaching Moment” — ARP poisoning in a real attack targets a specific interface. In your case it went to the wrong one, showing that Windows interface selection matters.

We can fix that using **netsh** commands (later in the presentation)

```
C:\WINDOWS\system32\CMD. X
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\97252>arp -a

Interface: 192.168.1.5 --- 0xe
Internet Address      Physical Address      Type
192.168.1.1           54-c2-50-21-1e-50    dynamic
192.168.1.4           fc-b0-de-d9-60-37    static
192.168.1.255         ff-ff-ff-ff-ff-ff    static
224.0.0.22            01-00-5e-00-00-16    static
224.0.0.251           01-00-5e-00-00-fb    static
224.0.0.252           01-00-5e-00-00-fc    static
239.255.255.250       01-00-5e-7f-ff-fa    static
255.255.255.255       ff-ff-ff-ff-ff-ff    static

Interface: 192.168.56.1 --- 0xf
Internet Address      Physical Address      Type
192.168.56.255       ff-ff-ff-ff-ff-ff    static
224.0.0.22            01-00-5e-00-00-16    static
224.0.0.251           01-00-5e-00-00-fb    static
224.0.0.252           01-00-5e-00-00-fc    static
239.255.255.250       01-00-5e-7f-ff-fa    static
255.255.255.255       ff-ff-ff-ff-ff-ff    static
```

```
:\Windows\System32>arp -s 192.168.1.1 aa-bb-cc-dd-ee-ff

:\Windows\System32>arp -a

Interface: 192.168.1.5 --- 0xe
Internet Address      Physical Address      Type
192.168.1.1           54-c2-50-21-1e-50    dynamic
224.0.0.22            01-00-5e-00-00-16    static

Interface: 192.168.56.1 --- 0xf
Internet Address      Physical Address      Type
192.168.1.1           aa-bb-cc-dd-ee-ff    static
224.0.0.22            01-00-5e-00-00-16    static
```

```
Administrator: שורת הפקודה
C:\Windows\System32>arp -d

C:\Windows\System32>
```

ICMP flood — CMD Continuous Ping

Process – Network Stress Test

CMD command: ping -t -l 65500 192.168.1.1

(Flood the router with almost infinite pings)

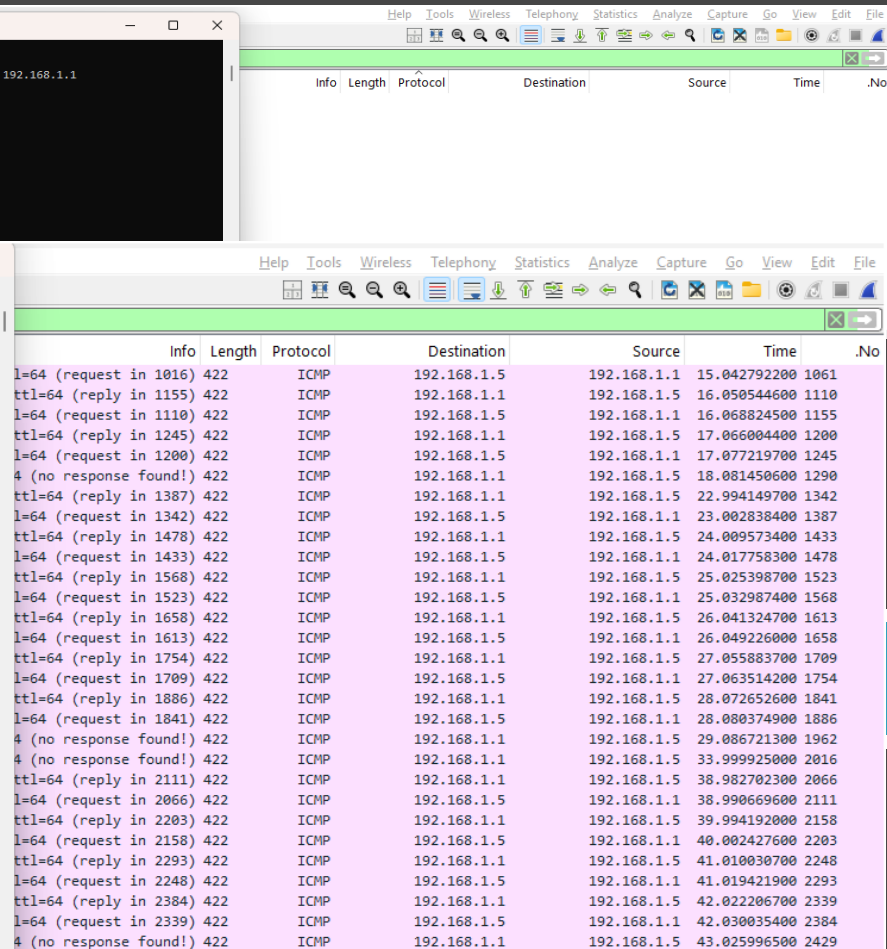
Result

Massive ping flood in the CMD and tons of icmp packets caught in wireshark

```
Administrator: שורת הפקודה
C:\Windows\System32>arp -d
C:\Windows\System32>ping -t -l 65500 192.168.1.1

t.
192.168.1.1: bytes=65500 time=9ms TTL=64
192.168.1.1: bytes=65500 time=7ms TTL=64
Reply from 192.168.1.1: bytes=65500 time=10ms TTL=64
Reply from 192.168.1.1: bytes=65500 time=9ms TTL=64
Request timed out.
Request timed out.
192.168.1.1: bytes=65500 time=9ms TTL=64
192.168.1.1: bytes=65500 time=7ms TTL=64
192.168.1.1: bytes=65500 time=8ms TTL=64
192.168.1.1: bytes=65500 time=8ms TTL=64
192.168.1.1: bytes=65500 time=10ms TTL=64
192.168.1.1: bytes=65500 time=8ms TTL=64
192.168.1.1: bytes=65500 time=8ms TTL=64
192.168.1.1: bytes=65500 time=9ms TTL=64
192.168.1.1: bytes=65500 time=11ms TTL=64
192.168.1.1: bytes=65500 time=7ms TTL=64
192.168.1.1: bytes=65500 time=18ms TTL=64
192.168.1.1: bytes=65500 time=11ms TTL=64
Request timed out.
Reply from 192.168.1.1: bytes=65500 time=8ms TTL=64
Reply from 192.168.1.1: bytes=65500 time=8ms TTL=64
Reply from 192.168.1.1: bytes=65500 time=7ms TTL=64
Reply from 192.168.1.1: bytes=65500 time=8ms TTL=64
Reply from 192.168.1.1: bytes=65500 time=7ms TTL=64
Reply from 192.168.1.1: bytes=65500 time=8ms TTL=64
Request timed out.
Request timed out.
Reply from 192.168.1.1: bytes=65500 time=8ms TTL=64
Reply from 192.168.1.1: bytes=65500 time=8ms TTL=64
Reply from 192.168.1.1: bytes=65500 time=9ms TTL=64
Reply from 192.168.1.1: bytes=65500 time=8ms TTL=64

Ping statistics for 192.168.1.1:
    Packets: Sent = 34, Received = 27, Lost = 7 (20% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 7ms, Maximum = 18ms, Average = 8ms
```

A screenshot of the Wireshark network protocol analyzer. The top pane shows the packet list with 422 entries, all of type ICMP. The middle pane shows the packet details for the selected packet, indicating it is an ICMP Echo (ping) request. The bottom pane shows the packet bytes. The interface is in Hebrew, with the title bar reading 'Administrator: שורת הפקודה' and the menu bar including 'Help', 'Tools', 'Wireless', 'Telephony', 'Statistics', 'Analyze', 'Capture', 'Go', 'View', 'Edit', and 'File'. The packet list table has columns for Info, Length, Protocol, Destination, Source, Time, and .No.

Info	Length	Protocol	Destination	Source	Time	.No
1=64 (request in 1016)	422	ICMP	192.168.1.5	192.168.1.1	15.042792200	1061
ttl=64 (reply in 1155)	422	ICMP	192.168.1.1	192.168.1.5	16.050544600	1110
1=64 (request in 1110)	422	ICMP	192.168.1.5	192.168.1.1	16.068824500	1155
ttl=64 (reply in 1245)	422	ICMP	192.168.1.1	192.168.1.5	17.066004400	1200
1=64 (request in 1200)	422	ICMP	192.168.1.5	192.168.1.1	17.077219700	1245
4 (no response found!)	422	ICMP	192.168.1.1	192.168.1.5	18.081450600	1290
ttl=64 (reply in 1387)	422	ICMP	192.168.1.1	192.168.1.5	22.994149700	1342
1=64 (request in 1342)	422	ICMP	192.168.1.5	192.168.1.1	23.002838400	1387
ttl=64 (reply in 1478)	422	ICMP	192.168.1.1	192.168.1.5	24.009573400	1433
1=64 (request in 1433)	422	ICMP	192.168.1.5	192.168.1.1	24.017758300	1478
ttl=64 (reply in 1568)	422	ICMP	192.168.1.1	192.168.1.5	25.025398700	1523
1=64 (request in 1523)	422	ICMP	192.168.1.5	192.168.1.1	25.032987400	1568
ttl=64 (reply in 1658)	422	ICMP	192.168.1.1	192.168.1.5	26.041324700	1613
1=64 (request in 1613)	422	ICMP	192.168.1.5	192.168.1.1	26.049226000	1658
ttl=64 (reply in 1754)	422	ICMP	192.168.1.1	192.168.1.5	27.055883700	1709
1=64 (request in 1709)	422	ICMP	192.168.1.5	192.168.1.1	27.063514200	1754
ttl=64 (reply in 1886)	422	ICMP	192.168.1.1	192.168.1.5	28.072652600	1841
1=64 (request in 1841)	422	ICMP	192.168.1.5	192.168.1.1	28.080374900	1886
4 (no response found!)	422	ICMP	192.168.1.1	192.168.1.5	29.086721300	1962
4 (no response found!)	422	ICMP	192.168.1.1	192.168.1.5	33.999925000	2016
ttl=64 (reply in 2111)	422	ICMP	192.168.1.1	192.168.1.5	38.982702300	2066
1=64 (request in 2066)	422	ICMP	192.168.1.5	192.168.1.1	38.990669600	2111
ttl=64 (reply in 2203)	422	ICMP	192.168.1.1	192.168.1.5	39.994192000	2158
1=64 (request in 2158)	422	ICMP	192.168.1.5	192.168.1.1	40.002427600	2203
ttl=64 (reply in 2293)	422	ICMP	192.168.1.1	192.168.1.5	41.010030700	2248
1=64 (request in 2248)	422	ICMP	192.168.1.5	192.168.1.1	41.019421900	2293
ttl=64 (reply in 2384)	422	ICMP	192.168.1.1	192.168.1.5	42.022206700	2339
1=64 (request in 2339)	422	ICMP	192.168.1.5	192.168.1.1	42.030035400	2384
4 (no response found!)	422	ICMP	192.168.1.1	192.168.1.5	43.025996500	2429

DNS Poisoning— Standard DNS

CMD: ipconfig /flushdns – clean dns cache

nslookup google.com 8.8.8.8 (Google dns)

nslookup google.com 1.1.1.1 (1.1.1.1 dns)

nslookup google.com 192.168.1.1 (my router's dns)

```
Control-C
C:\Windows\System32>ipconfig /flushdns
Windows IP Configuration
Successfully flushed the DNS Resolver Cache.
C:\Windows\System32>nslookup google.com 8.8.8.8
Server: dns.google
Address: 8.8.8.8
Non-authoritative answer:
Name: google.com
Addresses: 2a00:1450:4028:805::200e
142.250.75.78
C:\Windows\System32>nslookup google.com 1.1.1.1
Server: one.one.one.one
Address: 1.1.1.1
Non-authoritative answer:
Name: google.com
Addresses: 2a00:1450:4028:80a::200e
142.250.75.174
C:\Windows\System32>nslookup google.com 192.168.1.1
Server: Innbox684
Address: 192.168.1.1
Non-authoritative answer:
Name: google.com
Addresses: 2a00:1450:4028:80a::200e
142.250.75.110
Standard query response 0x00000000
Standard query response 0x231d
Standard query response 0xc237
Standard query response 0x00000000
C:\Windows\System32>
```

DNS Poisoning Detection Method:

Normal (what I captured):

8.8.8.8 → 142.250.75.78
1.1.1.1 → 142.250.75.174
Router → 142.250.75.110

All same range = Google
Load balancing = normal

Poisoned (what to look for):

8.8.8.8 → 142.250.75.78
1.1.1.1 → 142.250.75.174
Router → 185.220.101.47

Different range = POISONED
Router lying to you

Info	Length	Protocol	Destination	Source	Time	.No
ns2.goo	346	DNS	192.168.1.5	192.168.1.1	13.653306800	53
zoom.us	80	DNS	192.168.1.1	192.168.1.5	16.649407400	92
1137.aws	356	DNS	192.168.1.5	192.168.1.1	16.660453900	95
ddr.arpa	80	DNS	8.8.8.8	192.168.1.5	20.616906400	113
s.google	108	DNS	192.168.1.5	8.8.8.8	20.633553100	114
ogle.com	70	DNS	8.8.8.8	192.168.1.5	20.636523800	115
50.75.78	90	DNS	192.168.1.5	8.8.8.8	20.710887400	116
ogle.com	70	DNS	8.8.8.8	192.168.1.5	20.713919200	117
95::200e	102	DNS	192.168.1.5	8.8.8.8	20.781945200	118
ddr.arpa	80	DNS	1.1.1.1	192.168.1.5	25.908857800	121
one.one	113	DNS	192.168.1.5	1.1.1.1	25.925630800	122
ogle.com	70	DNS	192.168.1.5	1.1.1.1	25.926777100	123
8.75.174	90	DNS	192.168.1.5	1.1.1.1	25.939364600	124
ogle.com	70	DNS	192.168.1.5	1.1.1.1	25.942359500	125
9a::200e	102	DNS	192.168.1.5	1.1.1.1	25.954938600	126
ddr.arpa	84	DNS	192.168.1.1	192.168.1.5	33.031493600	143
nnbox684	107	DNS	192.168.1.5	192.168.1.1	33.034697800	144
ogle.com	70	DNS	192.168.1.1	192.168.1.5	33.036433800	145
3.google	334	DNS	192.168.1.5	192.168.1.1	33.049905100	146
ogle.com	70	DNS	192.168.1.1	192.168.1.5	33.054239300	147
oogle.c	346	DNS	192.168.1.5	192.168.1.1	33.065294700	148
sapp.com	76	DNS	192.168.1.1	192.168.1.5	61.334331600	216
sapp.com	76	DNS	192.168.1.1	192.168.1.5	61.334917100	217
sapp.com	89	DNS	192.168.1.1	192.168.1.5	61.336037000	218
sapp.com	76	DNS	192.168.1.1	192.168.1.5	61.336548400	219
sapp.net	158	DNS	192.168.1.5	192.168.1.1	61.349841000	220
.whatsap	372	DNS	192.168.1.5	192.168.1.1	61.349842100	221
.whatsap	372	DNS	192.168.1.5	192.168.1.1	61.349842900	222
sapp.net	146	DNS	192.168.1.5	192.168.1.1	61.349843700	223

DNS Poisoning– Poisoning the DNS

In order to change the DNS we need to manually edit the “hosts” document at “etc”

I added 1.2.3.4 as the IP address for Google

```
Administrator: שורת הפקודה
C:\Windows\System32>notepad C:\Windows\System32\drivers\etc\hosts
C:\Windows\System32>
```

```
hosts
ללא שם
קובץ ערוך הצג
# Copyright (c) 1993-2009 Microsoft Corp.
#
# This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
#
# This file contains the mappings of IP addresses to host names. Each
# entry should be kept on an individual line. The IP address should
# be placed in the first column followed by the corresponding host name.
# The IP address and the host name should be separated by at least one
# space.
#
# Additionally, comments (such as these) may be inserted on individual
# lines or following the machine name denoted by a '#' symbol.
#
# For example:
#
# 102.54.94.97 rhino.acme.com # source server
# 38.25.63.10 x.acme.com # x client host
#
# localhost name resolution is handled within DNS itself.
# 127.0.0.1 localhost
# ::1 localhost
1.2.3.4 google.com
```

DNS Poisoning– The DNS is Poisoned

*Curl google.com failed to connect to server

*powershell Resolve-DnsName returned the fake DNS address 1.2.3.4

*pinging Google failed because the system attempted to ping 1.2.3.4

*In wireShark we can see 4 ping requests and TCP requests that follows to try to find the match

```
C:\Windows\System32>notepad C:\Windows\System32\drivers\etc\hosts
C:\Windows\System32>curl http://google.com
curl: (28) Failed to connect to google.com port 80 after 21072 ms: Could not connect to server
C:\Windows\System32>powershell Resolve-DnsName google.com
Name                                     Type      TTL      Section   IPAddress
----
google.com                             A         51441    Answer    1.2.3.4

C:\Windows\System32>ping google.com
Pinging google.com [1.2.3.4] with 32 bytes of data:
Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 1.2.3.4:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
C:\Windows\System32>_
```

ip.addr == 1.2.3.4						
	Info	Length	Protocol	Destination	Source	Time
	Echo (ping) request id=0x0001, seq=74/18944, ttl=64 (no response found!)	74	ICMP	1.2.3.4	192.168.1.5	0.307694 2
	Echo (ping) request id=0x0001, seq=75/19200, ttl=64 (no response found!)	74	ICMP	1.2.3.4	192.168.1.5	5.122079 51
	Echo (ping) request id=0x0001, seq=76/19456, ttl=64 (no response found!)	74	ICMP	1.2.3.4	192.168.1.5	10.117457 65
	Echo (ping) request id=0x0001, seq=77/19712, ttl=64 (no response found!)	74	ICMP	1.2.3.4	192.168.1.5	15.112519 75
	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 15510	66	TCP	1.2.3.4	192.168.1.5	20.882074 81
	15510 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [TCP Retransmission]	66	TCP	1.2.3.4	192.168.1.5	21.891214 85
	15510 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [TCP Retransmission]	66	TCP	1.2.3.4	192.168.1.5	23.896380 145
	15510 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [TCP Retransmission]	66	TCP	1.2.3.4	192.168.1.5	27.899723 221
	15510 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [TCP Retransmission]	66	TCP	1.2.3.4	192.168.1.5	35.908366 274

NMAP – Part 1

The Command	The Result
nmap /?	to confirms nmap is installed
nmap-sn 192.168.1.0/24	Ping scan — discover live hosts, no port scanning Scan entire subnet — all 256 IPs from .0 to .255
nmap -sV	Service version detection — scan all ports AND identify what's running
nmap -p 80,443,22,21 192.168.1.1	Scan these particular port

1st — downloading nmap and running nmap /? To ensure nmap is installed

2nd — nmap -sn — 3 devices found in the network 192.168.1.0 (class /c Subnet Mask, from 0 to 255)

3rd — nmap -sV — scan all ports and see what's running

```
Administrator: שורת הפקודה - nmap -sV 192.168.1.1
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>nmap /?
Starting Nmap 7.80 ( https://nmap.org ) at 2026-05-29 10:57 ??? ????
Unable to split netmask from target expression: "/?"
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.77 seconds

C:\Windows\System32>nmap -sn 192.168.1.0/24
Starting Nmap 7.80 ( https://nmap.org ) at 2026-05-29 10:59 ??? ????
Nmap scan report for InnboxG84 (192.168.1.1)
Host is up (0.016s latency).
MAC Address: 54:C2:50:21:1E:50 (Unknown)
Nmap scan report for 192.168.1.4
Host is up (0.012s latency).
MAC Address: FC:B0:DE:D9:60:37 (Unknown)
Nmap scan report for 192.168.1.5
Host is up.
Nmap done: 256 IP addresses (3 hosts up) scanned in 7.61 seconds

C:\Windows\System32>nmap -sV 192.168.1.1
Starting Nmap 7.80 ( https://nmap.org ) at 2026-05-29 10:59 ??? ????
Stats: 0:00:19 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 0.00% done
```


NMAP – Part 2

• Nmap -p => scans relevant ports

• nmap results:

- 21/tcp filtered ftp ← FTP port exists but firewall blocking
- 22/tcp filtered ssh ← SSH port exists but firewall blocking
- 80/tcp open http ← router web interface accessible
- 443/tcp open https ← router secure web interface

• Wireshark output:

Tons of tcp requests triggered by nmap
filter: tcp.flags.syn==1 && !tcp.flags.ack==1
nmap TCP packets signals SACK_PERM

What is SACK_PERM?

SACK = Selective Acknowledgment Permission

It's a TCP option negotiated during the handshake:

Normal TCP (without SACK): Lost packet 5 of 10 sent Receiver: "I only got up to packet 4, resend from 4" Sender resends: 4, 5, 6, 7, 8, 9, 10 ← wastes bandwidth

TCP with SACK: Lost packet 5 of 10 sent Receiver: "I got 1-4 and 6-10, only missing 5" Sender resends: 5 only ← much more efficient

SACK_PERM in the SYN packet means: "I support Selective Acknowledgment — do you?"

It appears in nmap's SYN packets because nmap crafts its own TCP packets and includes standard TCP options to look like normal traffic and avoid detection. Every modern OS includes SACK_PERM in its SYN packets.

The Command	The Result
nmap /?	to confirms nmap is installed
nmap -sn 192.168.1.0/24	Ping scan — discover live hosts, no port scanning
nmap -sV	Service version detection — scan all ports AND identify what's running
nmap -p 80,443,22,21 192.168.1.1	Scan these particular port

```
C:\Windows\System32>nmap -p 80,443,22,21 192.168.1.1
Starting Nmap 7.80 ( https://nmap.org ) at 2026-05-29 11:03:11
Nmap scan report for Innbox684 (192.168.1.1)
Host is up (0.0016s latency).

PORT      STATE      SERVICE
21/tcp    filtered  ftp
22/tcp    filtered  ssh
80/tcp    open      http
443/tcp    open      https

MAC Address: 54:C2:50:21:1E:50 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 3.84 seconds
```

Time	Source	Destination	Protocol	Length	Info
184.043711	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 24100
184.056134	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 24101
184.060619	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 24102
184.074839	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 24103
184.080954	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 24104
184.084913	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 24105
184.089923	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 24106
184.104807	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 24107
184.115157	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 24108
184.135817	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 24109
184.157590	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 24110
184.164773	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 24111
184.204798	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 24112
184.209899	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 24113
184.370939	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 24114
184.386305	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 24115
184.418461	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 443 → 24116
184.418764	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 443 → 24117
184.467931	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 443 → 24118
185.313891	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 443 → 24119
185.432248	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 582
185.439667	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 443 → 260
185.443560	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 443 → 24120
185.441180	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 24121
185.443030	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 53 → 473
191.232917	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 443 → 24122
221.189397	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 443 → 24123
250.213726	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=1024 Len=0 MSS=1460 [SYN] 80 → 43880
250.217389	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=1024 Len=0 MSS=1460 [SYN] 21 → 43880
250.221155	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=1024 Len=0 MSS=1460 [SYN] 22 → 43880
250.222756	192.168.1.5	192.168.1.1	TCP	60	Seq=0 Win=1024 Len=0 MSS=1460 [SYN] 443 → 43880

OCSF - Certificate Inspection

curl -v
https://google.com 2>&1

The -v flag exposes everything that normally happens **invisibly** during an HTTPS connection — including the full TLS certificate exchange.

Every HTTPS connection triggers:

1. Certificate delivery (Google proves identity)
2. OCSF check (Windows verifies certificate valid)
3. Security headers (Google instructs browser behavior)

In Wireshark we see the certificate process in 1 line: server hello-certificate-key exchange-hello done

Wireshark packet capture showing an HTTPS connection to google.com. The packet list on the left shows a TCP segment (4235 bytes) from 192.168.1.5 to 13.89.179.13. The packet details pane shows the TLS handshake, including the Server Hello, Certificate, and Server Key Exchange. The packet bytes pane shows the raw data of the TLS handshake, including the Server Hello, Certificate, and Server Key Exchange.

Packet List:

No.	Time	Source	Destination	Protocol	Length	Info
389	203.033660	192.168.1.5	13.89.179.13	TLSv1.2	Done	Server Hello, Certificate, Server Key Exchange, Server Hello Done

Packet Details:

Server Hello, Certificate, Server Key Exchange, Server Hello Done

Packet Bytes:

```
* Connection #0 to host google.com:443 left intact
C:\Windows\System32\cmd.exe curl -v https://google.com 2>&1
* Host google.com:443 was resolved.
* IP6: <none>
* IP4: 142.250.75.174
* Trying 142.250.75.174:443...
* schannel: disabled automatic use of client certificate
* ALPN: curl offers http/1.1
* ALPN: server accepted http/1.1
* Established connection to google.com (142.250.75.174 port 443) from 192.168.1.5 port 59426
* using HTTP/1.x
* GET / HTTP/1.1
> Host: google.com
> User-Agent: curl/8.19.0
> Accept: */*
* Request completely sent off
* schannel: remote party requests renegotiation
* schannel: renegotiating SSL/TLS connection
* schannel: SSL/TLS connection renegotiated
* HTTP/1.1 301 Moved Permanently
Location: https://www.google.com/
Content-Type: text/html; charset=UTF-8
Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-src 'nonce-qI3PFwazWfqFteNoo_AxLA'
'strict-dynamic' 'report-sample' 'unsafe-eval' https://report-uri https://csp.withgoogle.com
csp/gws/other-hp
Date: Fri, 29 May 2026 08:22:39 GMT
Expires: Sun, 28 Jun 2026 08:22:39 GMT
Cache-Control: public, max-age=2592000
Server: gws
Content-Length: 220
X-XSS-Protection: 0
X-Frame-Options: SAMEORIGIN
Alt-Svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
<HTML><HEAD><meta http-equiv="content-type" content="text/html; charset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
<A href="https://www.google.com/">here</A>.
</BODY></HTML>
* Connection #0 to host google.com:443 left intact
```

Netsh - Firewall Rule Testing with netsh

Cmd: netsh advfirewall firewall add rule name="Block ICMP" protocol=icmpv4:8,any action=block dir=out

Netsh firewall now blocks ICMP protocol, and we can see the ping shows general failure

dir=in type=8Blocks incoming ping requests TO your PC

dir=out type=8Blocks your PC from sending pings OUT ← use this

advfirewall firewall **delete rule** name="Block ICMP"

ping 192.168.1.1 ← works again
Ping restored after rule deleted

```
C:\Windows\System32>netsh advfirewall firewall add rule name="Block ICMP" protocol=icmpv4:8,any action=block dir=out
Ok.
```

```
C:\Windows\System32>ping 192.168.1.1
```

```
Pinging 192.168.1.1 with 32 bytes of data:
General failure.
General failure.
General failure.
General failure.
```

```
Ping statistics for 192.168.1.1:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

```
C:\Windows\System32> netsh advfirewall firewall delete rule name="Block ICMP"
Deleted 1 rule(s).
Ok.
```

```
C:\Windows\System32>ping 192.168.1.1
```

```
Pinging 192.168.1.1 with 32 bytes of data:
Reply from 192.168.1.1: bytes=32 time=3ms TTL=64
Reply from 192.168.1.1: bytes=32 time=3ms TTL=64
Reply from 192.168.1.1: bytes=32 time=2ms TTL=64
Reply from 192.168.1.1: bytes=32 time=3ms TTL=64
```

```
Ping statistics for 192.168.1.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 2ms, Maximum = 3ms, Average = 2ms
```

Netsh - Firewall Rule Testing with netsh – Protocol Block List

<u>CMD command</u>	<u>What blocking it does</u>	<u>Type</u>	<u>Port</u>	<u>Protocol</u>
<i>netsh advfirewall firewall add rule name="Block DNS" protocol=udp localport=53 action=block dir=out</i>	<i>No domain name resolution — websites unreachable by name</i>	UDP	53	DNS
<i>netsh advfirewall firewall add rule name="Block HTTP" protocol=tcp localport=80 action=block dir=out</i>	<i>Blocks unencrypted web traffic</i>	TCP	80	HTTP
<i>netsh advfirewall firewall add rule name="Block HTTPS" protocol=tcp localport=443 action=block dir=out</i>	<i>Blocks encrypted web traffic — most internet stops working</i>	TCP	443	HTTPS
<i>netsh advfirewall firewall add rule name="Block FTP" protocol=tcp localport=21 action=block dir=out</i>	<i>Blocks file transfer control channel</i>	TCP	21	FTP
<i>netsh advfirewall firewall add rule name="Block SSH" protocol=tcp localport=22 action=block dir=out</i>	<i>Blocks secure remote connections and SFTP</i>	TCP	22	SSH
<i>netsh advfirewall firewall add rule name="Block Telnet" protocol=tcp localport=23 action=block dir=out</i>	<i>Blocks unencrypted remote connections</i>	TCP	23	Telnet
<i>netsh advfirewall firewall add rule name="Block SMTP" protocol=tcp localport=25 action=block dir=out</i>	<i>Blocks outgoing email sending</i>	TCP	25	SMTP
<i>netsh advfirewall firewall add rule name="Block NTP" protocol=udp localport=123 action=block dir=out</i>	<i>Breaks clock synchronization</i>	UDP	123	NTP
<i>netsh advfirewall firewall add rule name="Block ICMP" protocol=icmpv4: 8,any action=block dir=out</i>	<i>Blocks all ping requests outbound</i>	ICMP	—	ICMP (ping)
<i>netsh advfirewall firewall add rule name="Block RDP" protocol=tcp localport=3389 action=block dir=in</i>	<i>Blocks Windows Remote Desktop access</i>	TCP	3389	RDP
<i>netsh advfirewall firewall add rule name="Block SMB" protocol=tcp localport=445 action=block dir=in</i>	<i>Blocks Windows file sharing — seen in netstat as LISTENING</i>	TCP	445	SMB
<i>netsh advfirewall firewall add rule name="Block DHCP" protocol=udp localport=68 action=block dir=in</i>	<i>Blocks IP address assignment — PC gets no IP</i>	UDP	67/68	DHCP
<i>netsh advfirewall firewall add rule name="Block IMAP" protocol=tcp localport=143 action=block dir=out</i>	<i>Blocks incoming email retrieval</i>	TCP	143	IMAP
<i>netsh advfirewall firewall add rule name="Block LDAP" protocol=tcp localport=389 action=block dir=out</i>	<i>Blocks directory services — breaks domain login</i>	TCP	389	LDAP
<i>netsh advfirewall firewall add rule name="Block QUIC" protocol=udp localport=443 action=block dir=out</i>	<i>Forces browsers back to TCP — blocks Google's HTTP/3</i>	UDP	443	QUIC/HTTP3

Netsh - Firewall Rule Testing – ATTACKING THE SYSTEM

ATTACK BRIEF

□ HTTP/HTTPS Blocking

```
*netsh advfirewall firewall add rule name="Block HTTP"  
protocol=tcp remoteport=80 action=block dir=out  
*netsh advfirewall firewall add rule name="Block HTTPS"  
protocol=tcp remoteport=443 action=block dir=out
```

□ NTP Blocking

```
*netsh advfirewall firewall add rule name="Block NTP"  
protocol=udp localport=123 action=block dir=out
```

ARP poisoning

```
*netsh interface ipv4 add neighbors "Wi-Fi" 192.168.1.1  
aa-bb-cc-dd-ee-ff OR  
*netsh interface ipv4 add neighbors 14 192.168.1.1 aa-  
bb-cc-dd-ee-ff
```

DNS poisoning

```
*netsh advfirewall firewall add rule name="Block DNS"  
protocol=udp localport=53 action=block dir=out
```

Netsh - Firewall Rule Testing – BLOCKING HTTP/HTTPS

Administrator: שורת הפקודה

Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>netsh advfirewall firewall add rule name="Block HTTP" protocol=tcp remoteport=80 action=block dir=out
Ok.

C:\Windows\System32>curl example.com
curl: (7) Failed to connect to example.com port 80 after 162 ms: Could not connect to server

C:\Windows\System32>netsh advfirewall firewall add rule name="Block HTTPS" protocol=tcp remoteport=443 action=block dir=out
Ok.

C:\Windows\System32>curl https://google.com
curl: (7) Failed to connect to google.com port 443 after 19 ms: Could not connect to server

C:\Windows\System32>

Netsh - Firewall Rule Testing – BLOCKING NTP&ARP

Poisoning

BLOCKING THE NETWORK TIME PROTOCOL

```
C:\Windows\System32>netsh advfirewall firewall add rule name="Block NTP" protocol=udp localport=123 action=block dir=out
Ok.

C:\Windows\System32>w32tm /resync /force
The following error occurred: The service has not been started. (0x80070426)
```

ARP POISONING

```
C:\Windows\System32>netsh interface ipv4 add neighbors 14 192.168.1.1 aa-bb-cc-dd-ee-ff

C:\Windows\System32>arp -a

Interface: 192.168.1.5 --- 0xe
Internet Address      Physical Address      Type
192.168.1.1            aa-bb-cc-dd-ee-ff    static
192.168.1.3            72-b9-1d-ed-a0-0c    dynamic
192.168.1.4            fc-b0-de-d9-60-37    dynamic
192.168.1.6            aa-59-16-e5-84-94    dynamic
192.168.1.10           d4-f0-ea-86-87-8c    dynamic
192.168.1.255          ff-ff-ff-ff-ff-ff    static
224.0.0.22             01-00-5e-00-00-16    static
224.0.0.251            01-00-5e-00-00-fb    static
224.0.0.252            01-00-5e-00-00-fc    static
239.255.255.250        01-00-5e-7f-ff-fa    static
255.255.255.255        ff-ff-ff-ff-ff-ff    static
```

Note: 14 represents the interface number for the WI FI network

ARP POISONING

Ping to router fails due to wrong MAC number

```
C:\Windows\System32>ping 192.168.1.1

Pinging 192.168.1.1 with 32 bytes of data:
Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.1.1:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

Netsh - Firewall Rule Testing – DNS Poisoning

- 1) Using **netsh** to add firewall rule “**Block DNS**” (udp protocol, remote port 53)
- 2) **Flush** current saved DNS setting
- 3) **Pinging Google fails** “mysteriously” (DNS is disabled so no way to reach domain through name)

```
C:\Windows\System32>netsh advfirewall firewall add rule name="Block DNS" protocol=udp remoteport=53 action=block dir=out
Ok.
```

```
C:\Windows\System32>ipconfig /flushdns
```

```
Windows IP Configuration
```

```
Successfully flushed the DNS Resolver Cache.
```

```
C:\Windows\System32>ping google.com
```

```
Ping request could not find host google.com. Please check the name and try again.
```


Netsh — Fixing Netsh

- 1) netsh advfirewall reset – to reset netsh firewall changes.
- 2) Pinging google (by name) now works.
- 3) Arp table is fixed.

```
C:\Windows\System32>netsh advfirewall reset
Ok.

C:\Windows\System32>ping google.com

Pinging google.com [142.250.75.174] with 32 bytes of data:
Reply from 142.250.75.174: bytes=32 time=13ms TTL=115
Reply from 142.250.75.174: bytes=32 time=15ms TTL=115
Reply from 142.250.75.174: bytes=32 time=13ms TTL=115
Reply from 142.250.75.174: bytes=32 time=15ms TTL=115

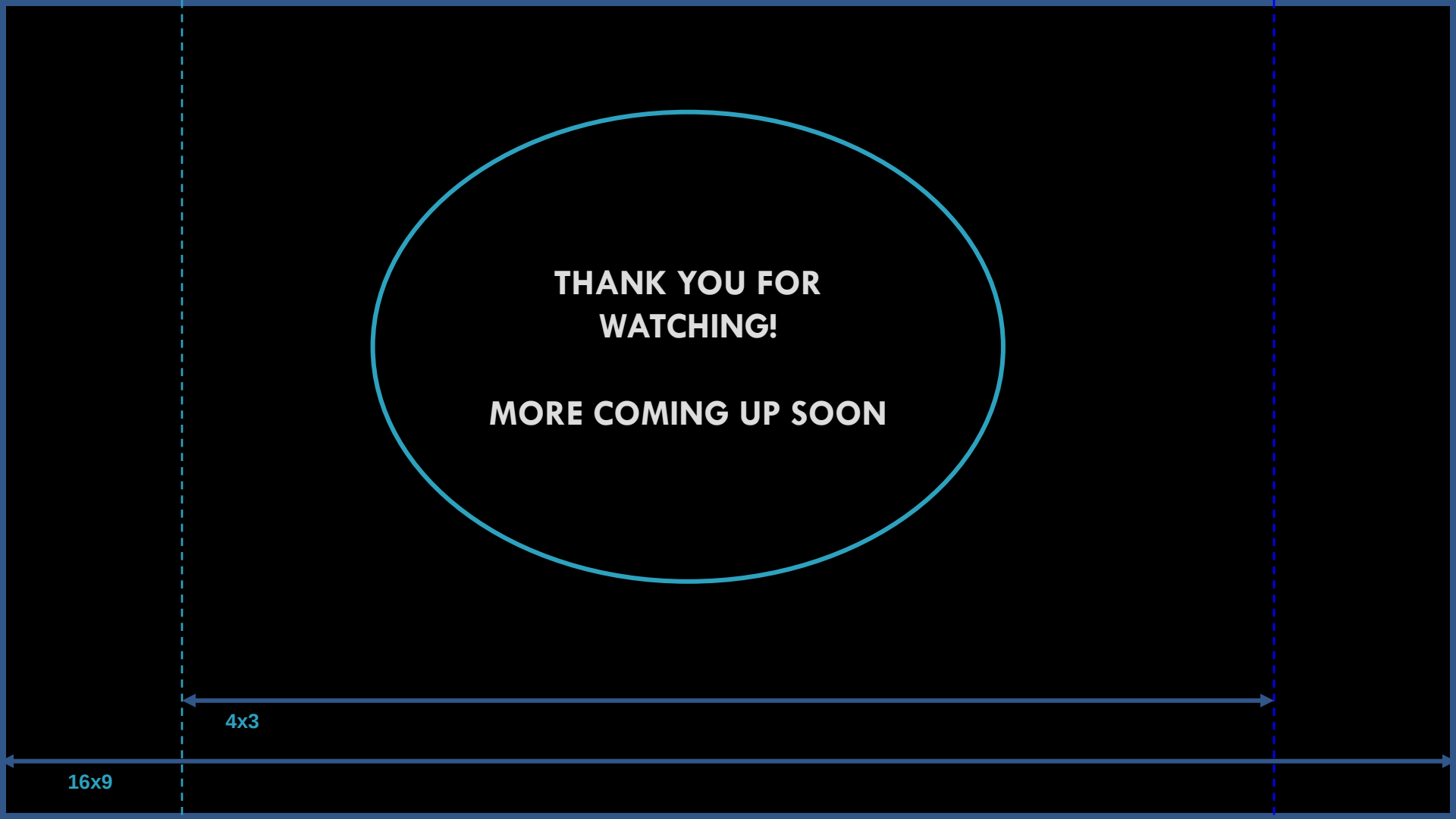
Ping statistics for 142.250.75.174:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 13ms, Maximum = 15ms, Average = 14ms

C:\Windows\System32>arp -a

Interface: 192.168.1.5 --- 0xe
    Internet Address      Physical Address          Type
    192.168.1.1            54-c2-50-21-1e-50        dynamic
    224.0.0.22             01-00-5e-00-00-16        static

Interface: 192.168.56.1 --- 0xf
    Internet Address      Physical Address          Type
    224.0.0.22             01-00-5e-00-00-16        static
    224.0.0.251            01-00-5e-00-00-fb        static

Interface: 169.254.134.230 --- 0x36
    Internet Address      Physical Address          Type
    224.0.0.22             01-00-5e-00-00-16        static
    224.0.0.251            01-00-5e-00-00-fb        static
    255.255.255.255        ff-ff-ff-ff-ff-ff        static
```



The diagram shows a black rectangular frame representing a 16x9 video screen. Two vertical dashed lines are positioned on the left and right sides of the frame. A horizontal double-headed arrow at the bottom spans the distance between these two dashed lines and is labeled '4x3'. A large, light blue oval is centered within the frame, containing the text 'THANK YOU FOR WATCHING!' and 'MORE COMING UP SOON' in white. The text is centered within the oval.

**THANK YOU FOR
WATCHING!**

MORE COMING UP SOON

4x3

16x9